

DOCUMENTAÇÃO DE PROJETO

AWS IAM

Introdução ao Identity and Access Management — Controle de Acesso por Função com o Princípio do Menor Privilégio

Parâmetro	Detalhe
Serviço principal	AWS IAM (Identity and Access Management)
Abordagem	Configuração de política de senhas, grupos, políticas e validação real de permissões
Cenário	Três novos profissionais com funções distintas em uma empresa em expansão na AWS
Duração aproximada	60 minutos

1. Problema / Contexto

Quando uma empresa começa a crescer na AWS, o acesso irrestrito à conta deixa de ser viável. Conceder permissões administrativas completas a todos os colaboradores é um risco grave: um usuário com acesso excessivo pode — intencionalmente ou não — modificar, excluir ou expor recursos que não fazem parte do seu escopo de trabalho.

O cenário deste projeto simulou uma situação real: uma empresa em expansão contratou três novos profissionais com funções distintas — suporte a armazenamento, suporte a infraestrutura de computação e administração de infraestrutura. Cada um precisava de um nível de acesso específico à AWS. O desafio era implementar esse controle de forma precisa, escalável e auditável, sem atribuir permissões individuais difíceis de gerenciar.

Além disso, a conta AWS não possuía nenhuma política de senhas configurada — um risco de segurança independente da política de acesso. Era necessário endereçar as duas frentes: a segurança das credenciais e o controle granular do que cada identidade pode fazer.

2. Objetivo

Meu objetivo técnico foi implementar um modelo de controle de acesso baseado em funções (RBAC) no AWS IAM, aplicando o princípio do menor privilégio a cada identidade. De forma específica:

- Configurar uma política de senhas com requisitos de complexidade e expiração para toda a conta AWS.
- Compreender a estrutura dos grupos, usuários e políticas pré-existentes no ambiente antes de qualquer alteração.
- Atribuir cada usuário ao grupo correto para que as permissões fossem herdadas por política de grupo — não por atribuição individual.

- Validar o modelo na prática fazendo login real com cada usuário e testando tanto as ações permitidas quanto as negadas, confirmando que o sistema se comportava exatamente como projetado.

3. Solução

A solução foi construída em três camadas interdependentes: segurança de credenciais (política de senhas), modelagem de acesso por função (grupos e políticas) e validação comportamental (testes reais com cada usuário). A terceira camada é a mais importante — não basta configurar corretamente, é preciso provar que o sistema nega o que deve negar tanto quanto permite o que deve permitir.

3.1 Política de Senhas da Conta

A primeira ação foi estabelecer uma política de senhas robusta para toda a conta. Essa configuração se aplica a qualquer usuário que criar ou redefinir uma senha — é a primeira linha de defesa contra comprometimento de credenciais.

Configuração	Valor definido
Comprimento mínimo	10 caracteres
Exigir letras maiúsculas	Ativado
Exigir letras minúsculas	Ativado
Exigir números	Ativado
Exigir caracteres especiais	Ativado
Expiração da senha	90 dias
Evitar reuso de senhas	Últimas 5 senhas
Reset por administrador ao expirar	Não ativado

Por que 90 dias e não menos: Políticas de expiração muito curtas forçam redefinições frequentes e incentivam usuários a escolher senhas previsíveis (ex.: adicionar um número ao final). O prazo de 90 dias equilibra renovação periódica com qualidade da senha — padrão amplamente adotado por frameworks de segurança como o CIS Benchmark.

3.2 Modelagem de Acesso por Função — Grupos e Políticas

Em vez de atribuir permissões diretamente a cada usuário, o modelo correto é criar grupos que representam funções e anexar políticas a esses grupos. O usuário herda as permissões do grupo ao ser adicionado a ele — e as perde automaticamente ao ser removido. Esse padrão torna o gerenciamento escalável: para onboarding de um novo colaborador na mesma função, basta adicioná-lo ao grupo existente.

Os três grupos já estavam pré-configurados no ambiente com suas respectivas políticas. Inspecionei cada um antes de qualquer atribuição:

Grupo EC2-Support**Política:** AmazonEC2ReadOnlyAccess (gerenciada pela AWS)*Permite listar e descrever instâncias EC2, ELB, CloudWatch e Auto Scaling. Não concede permissão para iniciar, parar ou modificar nenhum recurso.***Grupo S3-Support****Política:** AmazonS3ReadOnlyAccess (gerenciada pela AWS)*Permite listar buckets e visualizar objetos dentro deles. Não concede permissão para criar, editar, mover ou excluir nenhum objeto ou bucket.***Grupo EC2-Admin****Política:** EC2-Admin-Policy (política inline)*Concede permissão para descrever, iniciar e interromper instâncias EC2. É uma política pontual embutida diretamente no grupo — não reutilizável em outros grupos sem recriação manual.*

Toda política IAM é expressa como um documento JSON com três campos fundamentais:

Campo	Significado
Effect	Define se a regra concede (Allow) ou nega (Deny) o acesso
Action	Especifica a chamada de API afetada — ex.: ec2:DescribeInstances, s3:GetObject
Resource	Define o escopo: ARN de um recurso específico ou * para qualquer recurso

Exemplo de política mínima para parar instâncias EC2: `{ "Effect": "Allow", "Action": ["ec2:StopInstances", "ec2:DescribeInstances"], "Resource": "*" }`

3.3 Atribuição de Usuários aos Grupos

Com o mapeamento de grupos e políticas compreendido, adicionei cada usuário ao grupo correspondente à sua função. A atribuição foi feita via IAM > Grupos de usuários, selecionando o grupo de destino e adicionando o usuário pela guia Usuários.

Usuário	Grupo	Política	Escopo de acesso
user-1	S3-Support	AmazonS3ReadOnlyAccess	Leitura no S3 — sem acesso ao EC2
user-2	EC2-Support	AmazonEC2ReadOnlyAccess	Leitura no EC2 — sem parar instâncias
user-3	EC2-Admin	EC2-Admin-Policy (inline)	Descrever, iniciar e parar EC2

Após as três atribuições, verifiquei que cada grupo exibia exatamente um usuário na coluna correspondente — confirmando que nenhum usuário foi adicionado ao grupo errado e nenhum ficou sem grupo.

3.4 Validação Real das Permissões — Teste com Cada Usuário

A etapa mais criteriosa do projeto foi a validação comportamental: fazer login real com cada usuário em janela de navegação privada e testar sistematicamente as ações permitidas e negadas. Esse método é superior à simples inspeção de políticas no Console — ele confirma que o sistema está se comportando como configurado, não apenas que a configuração parece correta.

O URL de login dos usuários IAM foi obtido no painel do IAM Dashboard. Cada teste foi feito com a sessão de administrador fechada para evitar interferência de sessões ativas.

Usuário	Ação testada	Resultado esperado	Validado
user-1 (S3-Support)	Listar buckets S3	Permitido — leitura no S3	☑ Confirmado
user-1 (S3-Support)	Acessar EC2 > Instâncias	Negado — sem permissão EC2	☑ Confirmado
user-2 (EC2-Support)	Listar instâncias EC2	Permitido — leitura no EC2	☑ Confirmado
user-2 (EC2-Support)	Parar instância EC2	Negado — somente leitura	☑ Confirmado
user-2 (EC2-Support)	Acessar Amazon S3	Negado — sem permissão S3	☑ Confirmado
user-3 (EC2-Admin)	Parar instância EC2	Permitido — admin EC2	☑ Confirmado

Por que testar as negações é tão importante quanto testar as permissões: Um modelo de menor privilégio só está correto quando os dois lados funcionam: o usuário acessa o que deve e é bloqueado no que não deve. Testar apenas os acessos permitidos deixa metade da validação em aberto — e é exatamente nos acessos indevidos que erros de configuração se tornam riscos de segurança.

4. Ferramentas

AWS IAM	Políticas Gerenciadas AWS	Políticas Inline
Grupos de Usuários IAM	Política de Senhas IAM	Console de Gerenciamento AWS
JSON (estrutura de políticas)	EC2 (serviço de validação)	S3 (serviço de validação)

5. Resultado

O modelo de controle de acesso foi implementado e validado com sucesso. A política de senhas foi configurada e aplicada imediatamente a toda a conta. Os três usuários foram atribuídos aos grupos corretos e herdaram as permissões esperadas sem nenhuma atribuição individual.

A validação comportamental confirmou os seis cenários de teste — três permissões concedidas e três negações — todos operando exatamente como projetado. Cada mensagem de "acesso negado" foi, na prática, uma confirmação de que o sistema estava funcionando corretamente: o princípio do menor privilégio aplicado e verificável.

O resultado final foi um ambiente com identidades claramente separadas por função, permissões gerenciáveis em escala via grupos e credenciais protegidas por política de senha robusta — base mínima para qualquer conta AWS operando com responsabilidade.

6. Aprendizados

Este projeto tornou concreto um conceito que é frequentemente descrito em teoria mas raramente praticado de forma completa: o ciclo de configurar, testar e confirmar um modelo de controle de acesso. Os aprendizados mais relevantes vieram da diferença entre o que parece correto e o que realmente funciona.

6.1 Atribuição por grupo é a base da escalabilidade no IAM

Atribuir permissões diretamente a usuários individuais cria um problema de manutenção que cresce com o tamanho da equipe: para alterar o escopo de acesso de uma função, seria necessário editar cada usuário individualmente. Com grupos, a mudança é feita uma vez na política do grupo e propagada automaticamente para todos os membros.

O caso concreto foi claro: se amanhã a empresa contratar cinco novos profissionais de suporte a EC2, bastaria adicioná-los ao grupo EC2-Support. Nenhuma política nova. Nenhuma revisão de permissões individuais. O modelo já está pronto.

6.2 Política gerenciada vs. política inline — a diferença importa na manutenção

Os grupos EC2-Support e S3-Support usavam políticas gerenciadas pela AWS. O grupo EC2-Admin usava uma política inline. Na prática, a diferença ficou evidente:

Política gerenciada (AWS)

Criada de forma independente do grupo. Pode ser reutilizada em múltiplos grupos. Quando a AWS a atualiza (por exemplo, para incluir novos recursos do EC2), a mudança se propaga automaticamente para todos os grupos que a utilizam.

Política inline

Embutida diretamente no grupo ou usuário. Não é reutilizável. Se a mesma política precisar ser aplicada a outro grupo, deve ser recriada manualmente. Qualquer atualização exige edição em cada local onde foi definida.

Para ambientes em crescimento, políticas gerenciadas são sempre a escolha preferível. Políticas inline fazem sentido apenas para casos extremamente pontuais onde a política não deve ser compartilhada nem reutilizada.

6.3 O princípio do menor privilégio não é restrição — é proteção

Durante os testes, cada mensagem de "You are not authorized to perform this operation" pareceu, à primeira vista, um bloqueio. Mas é exatamente o oposto: é o sistema protegendo um recurso contra uma ação que aquele usuário não deveria ter. Um usuário de suporte a S3 que consegue parar instâncias EC2 não é mais capaz — é um risco.

Antipadrão comum: Conceder permissões AdministratorAccess a usuários por conveniência durante a configuração inicial e "ajustar depois". Na prática, o ajuste raramente acontece — e o usuário opera com acesso irrestrito indefinidamente. A configuração correta do IAM deve preceder o primeiro acesso, não ser postergada.

6.4 Validação comportamental é insubstituível

Inspecionar uma política no Console IAM mostra o que está configurado. Fazer login com o usuário e testar as ações mostra o que realmente acontece. Essa diferença importa porque permissões podem ser herdadas de múltiplas fontes — grupos, políticas inline, políticas de controle de serviço — e o resultado final nem sempre é intuitivo apenas pela leitura dos documentos JSON.

O método correto de validação de IAM sempre inclui testar com o usuário real. Qualquer outra abordagem deixa margem para surpresas em produção — e surpresas de acesso em produção costumam ser incidentes de segurança.